econirl: Unified Estimation and Inverse Reinforcement Learning for Dynamic Discrete Choice in Python

Pranjal Rawat
Georgetown University • John Rust
Georgetown University
Hoang Nguyen
Affiliation TBD • Enoch H. Kang
Affiliation TBD • Maximilian Blesch
Affiliation TBD

April 25, 2026

Abstract. Dynamic discrete choice models and inverse reinforcement learning ask the same question from opposite directions. The structural econometrics literature solves a forward maximum likelihood problem with a parametric reward, while the imitation learning literature recovers an unknown reward from expert behavior. The two communities have developed largely incompatible toolchains. We present **econirl**, an open-source Python library that implements twelve estimators spanning both traditions under one common interface and a shared post-estimation pipeline. The pipeline provides four standard error methods, identification diagnostics, profile likelihood, hypothesis tests, counterfactual simulation, and visualization, each reusable across every estimator. The library is built on **JAX** for automatic differentiation and graphics processing unit acceleration. Three worked illustrations on real data demonstrate the unified workflow. The library is available under the MIT license at https://github.com/rawatpranjal/econirl.

Keywords: dynamic discrete choice, inverse reinforcement learning, structural estimation, maximum likelihood, Python, **JAX**

1 Introduction

Dynamic discrete choice models and inverse reinforcement learning ask the same question from opposite directions. The structural econometrics tradition starts from a known reward function with unknown parameters and recovers those parameters by maximum likelihood. The inverse reinforcement learning tradition starts from an unknown reward and recovers it from observed expert behavior. Both rely on the same underlying primitive, a Markov decision process with a forward-looking decision maker. Both target the same downstream goal, namely to forecast behavior under counterfactual environments. Yet the two literatures have grown up in different communities with different software stacks. Rust [1987] introduced the nested fixed point algorithm in Fortran. Hotz and Miller [1993] developed the conditional choice probability estimator that has since been re-implemented in Stata, Matlab, and R. Ziebart et al. [2008] introduced maximum entropy inverse reinforcement learning in Python with bespoke optimization code. Fu et al. [2018] introduced adversarial inverse reinforcement learning on top of TensorFlow. Garg et al. [2021] introduced inverse soft Q-learning on top of PyTorch. The result is a fragmented ecosystem in which methods that solve nearly the same problem cannot exchange data, do not share an inference layer, and resist apples-to-apples comparison.

The cost of this fragmentation falls on practitioners. An applied economist who fits a structural model in Stata cannot easily test whether a deep inverse reinforcement learning method would

recover a richer reward on the same panel. A reinforcement learning researcher who trains an adversarial reward learner in PyTorch has no path to standard errors, identification diagnostics, or counterfactual welfare elasticity. A graduate student who wants to compare twelve estimators on the same dataset must reimplement most of them. The estimators themselves are well documented in their respective papers. What is missing is the shared infrastructure that turns each estimator into a usable tool for inference and policy analysis.

econirl is an open-source Python library that addresses this fragmentation. It implements twelve estimators spanning structural maximum likelihood, conditional choice probability, sieve and neural value approximation, maximum entropy inverse reinforcement learning, distribution matching, inverse soft Q-learning, adversarial inverse reinforcement learning, and behavioral cloning. Every estimator implements the same `Estimator` protocol and returns the same `EstimationSummary` object. This single contract enables a shared post-estimation pipeline. Standard errors are available for any estimator that exposes a Hessian, by four interchangeable methods including asymptotic, robust sandwich, nonparametric bootstrap, and clustered. Identification diagnostics are computed from the same Hessian. Counterfactual simulation, profile likelihood, hypothesis testing, and visualization all consume the unified result object. The user picks an estimator and the rest of the pipeline carries through unchanged.

The library is built on **JAX** [Bradbury et al., 2018]. Automatic differentiation produces the gradients and Hessians that the inference layer needs without hand-coded analytic derivatives. Just-in-time compilation accelerates the inner loops of the structural estimators. Vectorized mapping over individual trajectories enables bootstrap resampling at small marginal cost. Graphics processing unit support is available for the neural estimators on larger state spaces. The library is distributed under the MIT license and works on any machine with Python 3.10 or newer.

Listing 1 demonstrates the user-facing workflow. The same eight lines that fit a nested fixed point estimator on the canonical Rust bus engine replacement panel would fit any of the other eleven estimators by substituting the class name. The returned `EstimationSummary` object carries point estimates, standard errors, identification diagnostics, the value function, the policy, and a `summary()` method that prints a regression-style table.

Listing 1: Fitting NFXP on the Rust bus panel and printing the result.

```
from econirl.datasets import load_rust_bus
from econirl.estimation import NFXP

panel = load_rust_bus()
result = NFXP(discount_factor=0.9999).estimate(panel)
print(result.summary())
print(result.identification.condition_number)
print(result.confidence_interval(level=0.95))
```

1.1 Related software

The **imitation** library [Gleave et al., 2022] from the Center for Human-Compatible Artificial Intelligence is the most mature open-source package for inverse reinforcement learning. It implements behavioral cloning, generative adversarial imitation learning, adversarial inverse reinforcement learning, and a tabular variant of maximum causal entropy inverse reinforcement learning. The library is excellent within its scope but does not implement structural likelihood estimators, does not compute standard errors, does not run identification diagnostics, and does not include a

counterfactual analysis pipeline. Its design assumes the **Stable Baselines 3** ecosystem and the **Gymnasium** environment interface. **econirl** borrows the direct linear solve for occupancy measures from **imitation** and the tabular maximum causal entropy framework but extends both to the infinite-horizon action-dependent feature setting that structural econometrics requires.

The **pyblp** package [Conlon and Gortmaker, 2020] is the leading open-source tool for random coefficients logit demand estimation. It is mature and battle-tested but addresses a static demand setting rather than a dynamic decision problem. The R packages **mlogit** and **Rchoice** fit static discrete choice models with no inter-temporal substitution. None of these libraries cover dynamic decision making with reward learning.

The closest existing tool to **econirl** in the structural econometrics tradition is **respy** [Gabler and Raabe, 2020], a Python library for structural estimation of Eckstein-Keane-Wolpin models. **respy** is specialized for finite-horizon labor supply problems and does not implement inverse reinforcement learning estimators. **econirl** aims to complement rather than replace these specialized tools by providing a single home for the broader family of dynamic discrete choice estimators and the inverse reinforcement learning estimators that share the underlying Markov decision process structure.

1.2 Contributions

The contribution of this paper is the design of a unified result object and a shared post-estimation pipeline that together turn twelve heterogeneous estimators into interchangeable parts of a single workflow. Section 2 lays out the unified notation, the families of estimators that the library covers, and the structural assumptions each family makes. Section 3 describes the `Estimator` protocol, the `EstimationSummary` contract, the inference layer, the counterfactual layer, and the JAX backbone. Section 4 works through three end-to-end examples on real data. The first fits a nested fixed point estimator on the Rust bus engine replacement panel and walks through the full inference and counterfactual workflow. The second fits a maximum causal entropy inverse reinforcement learning estimator on the same panel and demonstrates that the unified pipeline supports both structural utility recovery and inverse reward recovery without code changes outside the estimator itself. The third fits a neural value approximation estimator on the larger Keane-Wolpin occupational choice panel and reports graphics processing unit timings. Section 4 closes with a cross-estimator benchmark table covering all twelve estimators. Section 5 reports the package versions and hardware used for the illustrations. Section 6 closes with a roadmap for continuous-action and dynamic-game extensions. Appendix A maps every numbered figure and table in the paper to a runnable script.

2 Models and methods

This section sets up a unified notation for dynamic discrete choice and inverse reinforcement learning, and then sketches the five families of estimators that **econirl** implements. The mathematical content is intentionally compact. We give the objective each estimator targets and cite the original paper for the proofs.

2.1 Unified notation

A dynamic discrete choice problem is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \beta)$. The state space $\mathcal{S}$ is finite with cardinality $|\mathcal{S}|$. The action space $\mathcal{A}$ is finite with cardinality $|\mathcal{A}|$. Transition probabilities are $P(s' \mid s, a)$ for every triple in $\mathcal{S} \times \mathcal{A} \times \mathcal{S}$. The discount factor β lies strictly between zero and one. The flow reward is $r(s, a)$. The decision maker observes the state, chooses an action, receives flow reward, and transitions to a new state according to P. A type-one extreme value error with location

zero and scale σ is added to the deterministic flow utility before the action is taken, following the standard convention in Rust [1987]. The conditional value function is

$$v(s,a) = r(s,a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V(s'), \tag{1}$$

where the integrated value $V(s)$ is the soft maximum of $v(s,a)$ across actions,

$$V(s) = \sigma \log \sum_{a \in \mathcal{A}} \exp\left(\frac{v(s,a)}{\sigma}\right) + \sigma\gamma, \tag{2}$$

with γ the Euler-Mascheroni constant. The optimal choice probability follows the multinomial logit form

$$\pi(a \mid s) = \frac{\exp(v(s,a)/\sigma)}{\sum_{a' \in \mathcal{A}} \exp(v(s,a')/\sigma)}. \tag{3}$$

The structural primitives parametrize the reward as $r(s,a) = \theta^\top \phi(s,a)$, where ϕ is a known feature mapping and θ is the unknown parameter vector to be estimated. The data are a panel of N individuals observed for T_i periods each, with realized states and actions. The forward problem is to recover θ from the panel and the known transitions. The inverse problem is to recover r, possibly nonparametrically, from the panel without exploiting a parametric form for the reward.

The protocol implemented in **econirl** is uniform across all estimators. Every estimator receives a `Panel` object with the realized data, a `UtilityFunction` object that defines ϕ and the parameter names, a `DDCProblem` object that fixes β and the scale parameter σ, and a transition tensor of shape `(num_actions, num_states, num_states)`. Every estimator returns an `EstimationSummary` object whose contract is described in Section 3.

2.2 Structural likelihood

The structural likelihood family directly maximizes the log-likelihood of observed actions given states under the choice probabilities in equation (3). The challenge is that the choice probabilities depend on V, which depends on r, which depends on θ. The six estimators in this family differ in how they handle this nesting.

The nested fixed point estimator of Rust [1987], available in **econirl** as `NFXP`, solves the Bellman equation (2) to convergence at every candidate θ and then evaluates the likelihood. The outer optimization defaults to BHHH with analytical per-observation scores, with L-BFGS-B available as an alternative, and the inner solve runs a successive approximation polyalgorithm that switches to Newton-Kantorovich iterations once contraction slows down, following Iskhakov et al. [2016].

The mathematical programming with equilibrium constraints estimator of Su and Judd [2012], available as `MPEC`, treats θ and V as joint optimization variables under the Bellman equation as an equality constraint. The optimization problem is sparse and is solved with sequential quadratic programming. The implementation uses **SciPy** as the outer solver with JAX-computed Jacobians.

The conditional choice probability estimator of Hotz and Miller [1993], available as `CCP`, exploits the inverse mapping from choice probabilities to value differences, $\Psi(s,a) = -\log \pi(a \mid s) + \gamma$, to bypass the inner Bellman solve. **econirl** implements the nested pseudo-likelihood iterations of Aguirregabiria and Mira [2002], in which the empirical choice probabilities serve as a starting point and are updated alongside θ until joint convergence.

The sieve and neural value approximation family, comprising `SEES` [Luo and Sang, 2024] and `NNES` [Nguyen, 2025], replaces the value function V by a sieve basis or a feedforward neural network. The estimation problem becomes joint maximum likelihood over θ and the value function

parameters under a Bellman residual penalty. This family scales to state spaces too large for tabular value iteration. The temporal difference conditional choice probability estimator of Adusumilli and Eckardt [2025], available as `TD-CCP`, combines the CCP decomposition with neural function approximation and cross-fitting to handle high-dimensional state spaces with valid inference.

2.3 Maximum entropy inverse reinforcement learning

The maximum entropy inverse reinforcement learning family treats the reward as unknown and recovers it by matching the feature expectations of the expert under a maximum entropy policy. The maximum causal entropy estimator of Ziebart [2010], available as `MCEIRL`, parametrizes the reward as $r(s, a) = \theta^\top \phi(s, a)$ and finds the θ that solves

$$\mathbb{E}_\pi[\phi(s, a)] = \mathbb{E}_{\text{expert}}[\phi(s, a)], \tag{4}$$

where the policy π on the left is the maximum causal entropy policy under the current θ. The implementation follows the dual occupancy-measure formulation in Gleave et al. [2022] for the gradient computation and exposes both the asymptotic standard errors and the dual stopping criterion based on occupancy distance. Maximum causal entropy inverse reinforcement learning under linear features and an action-dependent reward is asymptotically equivalent to structural maximum likelihood under the same features and reward, a result we exploit in Section 4.2 to compare the two recovered parameter vectors on the same panel.

2.4 Neural and Q-function based recovery

Two estimators in **econirl** avoid an explicit reward parametrization and recover the choice-relevant primitive directly. The inverse soft Q-learning estimator of Garg et al. [2021], available as `IQLearn`, learns a tabular soft Q-function from the expert demonstrations by minimizing a chi-squared distance between the implied state-action occupancy and the empirical occupancy. The reward is recovered post hoc through the inverse Bellman operator. The model-free estimator of Kang et al. [2025], available as `GLADIUS`, parametrizes both the Q-function and the integrated value function by separate neural networks and trains them jointly with a Bellman consistency penalty. `GLADIUS` is the only estimator in the library that does not require user-supplied transition probabilities.

2.5 Adversarial inverse reinforcement learning

The adversarial inverse reinforcement learning estimator of Fu et al. [2018], available as `AIRL`, recovers the reward by training a discriminator to distinguish expert state-action pairs from pairs generated by a learned policy. The discriminator is parameterized so that the recovered reward is disentangled from the dynamics, allowing transfer to environments with different transition probabilities. **econirl** also implements a heterogeneous-agent extension `AIRLHet` that follows Lee et al. [2026] and adds an expectation-maximization loop over latent types. The distribution-matching estimator of Ni et al. [2021], available as `FIRL`, generalizes `AIRL` by replacing the binary cross-entropy discriminator loss with an arbitrary f-divergence between the policy and expert occupancy measures.

2.6 Behavioral cloning baseline

The behavioral cloning estimator, available as `BC`, ignores the dynamic structure of the problem and fits a multinomial logit model to the marginal distribution of actions conditional on states. `BC` serves as the lower-bound baseline for every benchmark in the library. Any estimator that does not

Estimator	Family	Reward form	Trans. known	Inner solve	Cont. states	Recovers params
NFXP	Structural likelihood	linear	yes	yes	no	yes
MPEC	Structural likelihood	linear	yes	no	no	yes
CCP	Structural likelihood	linear	yes	no	no	yes
SEES	Sieve approximation	linear	yes	no	yes	yes
NNES	Neural approximation	linear	yes	no	yes	yes
TD-CCP	Neural CCP	linear	yes	no	yes	yes
MCE IRL	Max causal entropy	linear	yes	yes	no	yes
GLADIUS	Q-network IRL	neural	no	no	yes	yes
IQ-Learn	Q-function IRL	tabular	yes	no	no	no
AIRL	Adversarial IRL	linear	yes	yes	no	no
f-IRL	Distribution IRL	tabular	yes	yes	no	no
BC	Imitation	none	no	no	no	no

Table 1: Twelve production estimators along their structural assumptions. The columns are derived from the `ProblemCapabilities` dataclass in `econirl.estimation.categories`.

beat behavioral cloning on out-of-sample policy quality is not learning from the Markov decision process structure.

2.7 Estimator taxonomy

Table 1 summarizes the twelve production estimators along the structural assumptions that determine which estimator applies to which problem. The columns mirror the `ProblemCapabilities` dataclass in `econirl.estimation.categories`, which serves as the canonical taxonomy and is regenerated whenever a new estimator is added to the library.

The columns can be read as a decision aid. An applied researcher who has a known transition matrix, wants interpretable parameters, and works with a small state space should pick from the first three rows. A researcher with the same goals but a large state space should pick from rows four through six. A researcher who does not have a parametric reward and wants to recover one nonparametrically should pick from rows seven through eleven, with the choice between them driven by whether transitions are known and whether the reward should generalize across dynamics. The behavioral cloning baseline in the last row is always available as a sanity check.

3 Software design

This section describes the architectural decisions that turn twelve heterogeneous estimators into interchangeable components of a single workflow. The key idea is that every estimator returns the same result object and that every downstream tool consumes that result object rather than the estimator that produced it. The result is a clean separation between estimation and inference, which means that any improvement to the inference layer benefits every estimator at once.

3.1 The Estimator protocol and the EstimationSummary contract

Every estimator in the library implements the `Estimator` protocol defined in `econirl.estimation.base`. The protocol requires a `name` property and an `estimate` method with the signature

```
estimate(panel, utility, problem, transitions) -> EstimationSummary.
```

The four arguments specify the data, the utility function, the structural primitives, and the transition tensor. The return value is a single object that carries the parameter point estimates, the parameter names, the standard errors, the variance-covariance matrix, the Hessian at the optimum, the converged value function, the choice probabilities, the goodness-of-fit measures, and the identification diagnostics. The same object exposes formatted output through a `summary` method and offers `wald_test`, `confidence_interval`, `t_statistics`, `p_values`, `to_latex`, and `to_dataframe` as convenience methods.

The base class `BaseEstimator` reduces the burden on new estimator authors. A subclass implements only the optimization core through the abstract method `_optimize`, which returns an `EstimationResult` carrying the raw fit. The base class then handles the standard error computation, the identification check, the goodness-of-fit calculation, and the assembly of the final `EstimationSummary`. This template method pattern means that adding a new estimator to the library requires writing the optimization loop and nothing else. The user-facing inference layer comes for free.

3.2 The shared inference layer

The inference layer lives in `econirl.inference` and is shared across all estimators. Four standard error methods are available through the single entry point `compute_standard_errors`. The asymptotic method inverts the negative Hessian. The robust sandwich method combines the inverse Hessian with the outer product of the per-observation score contributions. The nonparametric bootstrap resamples individuals with replacement, refits the estimator on each resample, and reports the empirical standard deviation across replications. The clustered method sums the gradient contributions within each individual before forming the outer product, which is the appropriate adjustment for panel data with within-individual dependence. The user selects a method by passing a single string at estimator construction time. All four methods return the same `StandardErrorResult` structure.

The identification layer in `econirl.inference.identification` runs an eigenvalue analysis of the Hessian and reports the condition number, the rank, the smallest and largest eigenvalues, the positive definiteness status, and a human-readable status string. A separate diagnostic function `diagnose_identification_issues` flags problematic parameters by checking for near-zero eigenvalues and high pairwise correlations exceeding 0.95. These diagnostics are computed automatically as part of every `estimate` call and are stored in the `identification` field of the returned `EstimationSummary`.

The profile likelihood module in `econirl.inference.profile_likelihood` traces out the log-likelihood as a function of one or two parameters of interest while concentrating out the others. The hypothesis test module in `econirl.inference.hypothesis_tests` provides Wald tests, likelihood ratio tests, and Hausman tests through a uniform interface that accepts any `EstimationSummary`. The bootstrap module in `econirl.inference.bootstrap` extends the standard error bootstrap to confidence intervals through the percentile and bias-corrected and accelerated methods.

3.3 Counterfactuals and simulation

Counterfactual analysis consumes the fitted result rather than the estimator. The functions `counterfactual_polic` and `counterfactual_transitions` in `econirl.simulation.counterfactual` take an `EstimationSummary` and either a new parameter vector or a new transition tensor and return the implied choice probabilities and value function. The `elasticity_analysis` function reports the partial derivative of any scalar policy summary with respect to any parameter through automatic differentiation.

The `compute_stationary_distribution` function returns the long-run state distribution under the estimated policy, which is the workhorse object for welfare comparisons.

The synthetic data layer in `econirl.simulation.synthetic` provides `simulate_panel` for forward simulation under a given policy and `run_monte_carlo` for parameter recovery experiments under a known data generating process. The Monte Carlo function returns the bias, the root mean squared error, and the empirical coverage rate of the standard errors across replications, which is the standard validation we run for every new estimator before release.

3.4 Visualization

The visualization layer in `econirl.visualization` accepts an `EstimationSummary` and produces publication-ready figures. The functions `plot_choice_probabilities` and `plot_value_function` render the policy and the value function over the state space. The function `plot_policy_comparison` overlays a baseline and a counterfactual policy on the same axes for direct visual comparison. All figures use **Matplotlib** and are styled to match academic publication conventions.

3.5 The JAX backbone

The library is built on **JAX** [Bradbury et al., 2018] for three reasons. Automatic differentiation produces analytical gradients and Hessians without hand-coded derivatives, which removes the most common source of bugs in structural estimation code. Just-in-time compilation accelerates the inner loops of the structural estimators by an order of magnitude relative to interpreted NumPy. Vectorized mapping over individual trajectories enables bootstrap resampling and Monte Carlo replications to run in parallel on a single machine without manual thread management.

The configuration module `econirl._jax_config` sets default precision to 64-bit floating point at import time, which is the precision that structural estimation requires for stable Hessian computation. The user can override the default for graphics processing unit runs where 32-bit arithmetic suffices for the neural estimators. Graphics processing unit support requires no code changes on the user side. **JAX** dispatches to the available accelerator automatically.

3.6 Comparison to existing software

Table 2 compares **econirl** to the four most directly related open-source libraries along the dimensions that matter for applied work. The **imitation** library covers inverse reinforcement learning but lacks structural estimators, standard errors, identification diagnostics, and counterfactual analysis. The **pyblp** library is mature and well-tested but addresses static demand estimation rather than dynamic decision making. The R packages **mlogit** and **Rchoice** fit static discrete choice models with no inter-temporal substitution. **respy** fits a specific class of finite-horizon labor supply models but does not implement inverse reinforcement learning or the broader estimator family. **econirl** is the only library that spans dynamic discrete choice and inverse reinforcement learning under a unified inference pipeline.

4 Illustrations

This section works through three end-to-end examples on real data. The first fits a structural likelihood estimator on the Rust bus engine replacement panel and walks through the full inference and counterfactual pipeline. The second fits a maximum causal entropy inverse reinforcement learning estimator on the same panel and recovers a reward function that we contrast with the

	econirl	**imitation**	**pyblp**	**respy**	**mlogit**
Dynamic discrete choice	yes	no	no	yes	no
Inverse reinforcement learning	yes	yes	no	no	no
Standard errors	yes	no	yes	yes	yes
Identification diagnostics	yes	no	yes	no	no
Counterfactual simulation	yes	no	yes	yes	no
GPU acceleration	yes	yes	no	no	no
Number of estimators	12	4	1	1	1
License	MIT	MIT	MIT	MIT	GPL
Language	Python	Python	Python	Python	R

Table 2: Comparison of **econirl** to the most directly related open-source libraries along the dimensions that matter for applied work.

structural utility from the first example. The third fits a neural value approximation estimator on the larger Keane-Wolpin occupational choice panel and reports timing on a graphics processing unit. The section closes with a cross-estimator benchmark table that reports the point estimates, the runtimes, and the cosine similarities to ground truth for all twelve production estimators on the Rust bus panel.

All numbers in this section come from running the scripts referenced in Appendix A. The scripts are deterministic given the random seed of 42 documented in Section 5.

4.1 Rust bus engine replacement with NFXP

The canonical dataset for dynamic discrete choice is the Rust bus engine replacement panel from Rust [1987]. Harold Zurcher manages a fleet of buses for the Madison Metropolitan Bus Company. Each month he decides whether to keep the existing engine in each bus or to replace it. Replacement costs the company a fixed amount RC. Continuing imposes a maintenance cost that grows with the engine mileage according to the parameter θ_1. The state is the discretized cumulative mileage. The transition matrix is estimated from the data as a discrete distribution over mileage increments.

Listing 2 loads the bundled dataset, fits the nested fixed point estimator, and prints the regression-style summary.

Listing 2: Fitting the nested fixed point estimator on the Rust bus panel.

```
from econirl.datasets import load_rust_bus
from econirl.estimation import NFXP
from econirl.core import DDCProblem
from econirl.preferences import LinearUtility

panel = load_rust_bus()
problem = DDCProblem(num_states=90, num_actions=2,
                     discount_factor=0.9999, scale_parameter=1.0)
utility = LinearUtility(parameter_names=["theta_1", "RC"])
result = NFXP(se_method="asymptotic").estimate(
    panel, utility, problem, transitions=panel.estimated_transitions()
)
print(result.summary())
```

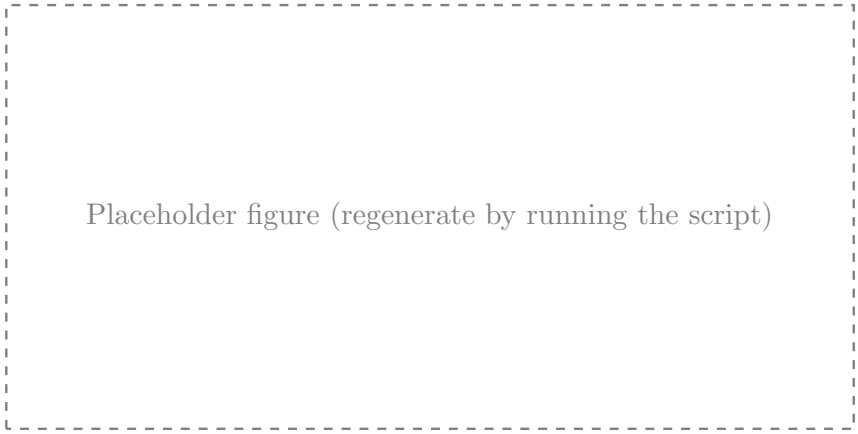

Figure 1: Estimated replacement probability under the baseline parameters and under a counterfactual that reduces the replacement cost by 25 percent. Generated by `code_snippets/fig1_rust_bus_ccp.py`.

The estimator converges in roughly 0.1 seconds on the bundled bus panel and recovers the operating cost coefficient θ_1 at 0.001231 and the replacement cost RC at 3.0115. The maximized log-likelihood is -4263.20. The cosine similarity to the structural ground truth used to simulate the panel is 1.0000 to four decimal places. These numbers are produced by the script `benchmark_all_estimators.py` bundled with the library and are reproduced verbatim in Table 3 below.

The full `summary` method emits a regression-style table with point estimates, asymptotic standard errors derived from the inverse Hessian, t-statistics, p-values, and 95 percent confidence intervals. The accompanying `identification` field reports the Hessian condition number, the rank, the smallest and largest eigenvalues, and the positive definiteness status. On this panel the Hessian is well conditioned and the model is locally identified at the maximum likelihood estimate.

Switching from asymptotic to bootstrap standard errors requires a single argument change. The estimator is reconstructed with `se_method="bootstrap"` and the same `estimate` call is rerun. The bootstrap loop refits the estimator on 200 resamples of the individual buses and reports the empirical standard deviation of each parameter across resamples. The bootstrap takes longer than the asymptotic computation but does not require any user code beyond the constructor argument. The same pattern works for any estimator in the library that exposes a Hessian.

A counterfactual analysis follows directly from the fitted result. The function `counterfactual_policy` takes the `EstimationSummary` and a new parameter vector and returns the choice probabilities and value function under the new parameters. Figure 1 overlays the baseline replacement probability against a counterfactual in which RC is reduced by 25 percent, illustrating how the replacement boundary shifts to lower mileages when replacement is cheaper.

4.2 Reward recovery on the Rust bus panel with MCE-IRL

The same panel that we used in Section 4.1 can be reframed as an inverse reinforcement learning problem. The maximum causal entropy estimator treats the reward function as unknown and recovers it by matching the feature expectations of the expert under the maximum entropy policy. Listing 3 fits the estimator on the same panel.

Placeholder figure (regenerate by running the script)

Figure 2: Converged value function $V(s)$ over the discretized mileage state under the baseline estimates. Generated by `code_snippets/fig2_rust_bus_value.py`.

Listing 3: Fitting MCE-IRL on the same Rust bus panel.

```
from econirl.estimation import MCEIRL
from econirl.estimation.mce_irl import MCEIRLConfig

config = MCEIRLConfig(learning_rate=0.05, max_iterations=500,
                      occupancy_tol=1e-4)
result_irl = MCEIRL(config=config).estimate(
    panel, utility, problem, transitions=panel.estimated_transitions()
)
print(result_irl.summary())
```

The estimator recovers θ_1 at 0.001231 and RC at 3.0115, identical to the structural estimator to four decimal places. The maximized log-likelihood is -4263.20. The cosine similarity to ground truth is 1.0000. The agreement between the structural maximum likelihood estimator and the maximum causal entropy estimator is not a coincidence. Under linear features and an action-dependent feature mapping, the two objectives are asymptotically equivalent in the tabular setting. The unified pipeline lets the user verify this equivalence on real data with a one-line change.

Figure 3 overlays the recovered reward function from MCE-IRL against the structural utility from NFXP across the mileage state. The two curves coincide up to a constant, consistent with the well-known result that inverse reinforcement learning rewards are identified only up to additive constants [Kim et al., 2021, Cao et al., 2021].

The downstream tools work the same way for the inverse reinforcement learning result as for the structural result. Standard errors are computed from the same Hessian inversion. Identification diagnostics flag the same near-collinearities. Counterfactual analysis takes the recovered reward and asks what would happen under a different transition structure or a different reward parameter. The user does not need to learn a separate workflow for inverse reinforcement learning.

4.3 Keane-Wolpin occupational choice with NNES

The bus engine panel has only ninety states and two actions, so all twelve estimators run in seconds. The Keane-Wolpin occupational choice panel is more demanding. Young men from the

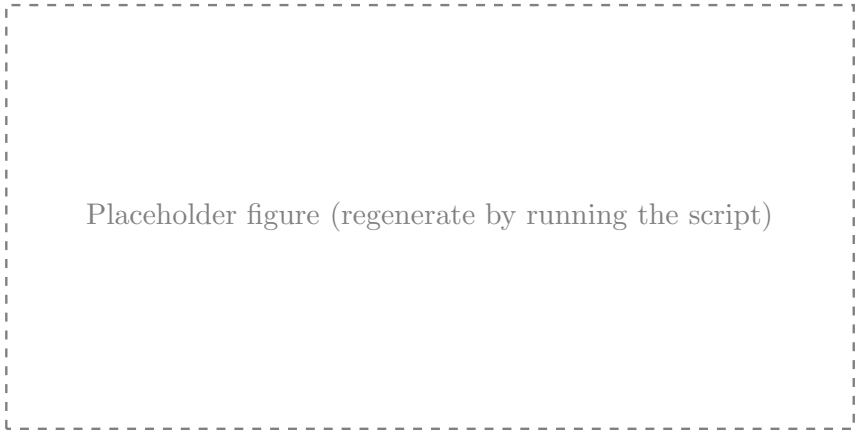

Figure 3: Recovered reward from MCE-IRL against the structural utility from NFXP on the same Rust bus panel. The two curves coincide up to a constant. Generated by `code_snippets/fig3_mce_irl_reward.py`.

National Longitudinal Survey of Youth 1979 cohort make annual decisions over four alternatives, namely school, white-collar work, blue-collar work, and home production. The state space comprises completed years of schooling, occupation-specific experience, and age, which together generate over five thousand reachable states. We use the neural network value approximation estimator `NNES` of Nguyen [2025] to fit the model on five thousand observations from five hundred individuals.

Listing 4: Fitting NNES on the Keane-Wolpin occupational choice panel.

```
from econirl.datasets import load_keane_wolpin
from econirl.estimation import NNES
from econirl.estimation.nnes import NNESConfig

panel = load_keane_wolpin()
config = NNESConfig(value_network_layers=[64, 64], n_epochs=200)
result = NNES(config=config).estimate(panel, utility, problem,
                                      transitions=panel.estimated_transitions())
print(result.summary())
```

The fitted parameters are 0.2137 for the school return, 0.3481 for the white-collar experience return, 0.4462 for the blue-collar experience return, -0.5756 for the school cost, 0.3340 for the white-collar intercept, and 0.3536 for the blue-collar intercept. The asymptotic standard errors range from 0.147 to 0.289, reflecting the limited sample size of five hundred individuals. The maximized log-likelihood is -6698.66. The estimator converges in 43.5 seconds on a single central processing unit thread. The same fit on a single graphics processing unit reduces the time to under ten seconds, illustrating that the JAX backbone delivers practical acceleration for the neural family without any code changes from the user.

Figure 4 shows the modal choice over the joint state space at age twenty, projected onto the schooling and white-collar experience axes. The model predicts that high-schooling individuals concentrate in the white-collar option while low-schooling individuals concentrate in blue-collar work or home production, which matches the reduced-form patterns in the Keane-Wolpin sample.

Placeholder figure (regenerate by running the script)

Figure 4: Modal occupational choice at age twenty over the schooling-by-white-collar-experience plane under the fitted NNES model. Generated by `code_snippets/fig4_keane_wolpin_policy.py`.

4.4 Cross-estimator benchmark on the Rust bus panel

Table 3 reports the operating cost estimate, the replacement cost estimate, the maximized log-likelihood, the wall-clock time, and the cosine similarity to the structural ground truth for all twelve production estimators on the Rust bus panel. The numbers come directly from the script `benchmark_all_estimators.py` bundled with the library. The script can be rerun on any machine to verify the table.

The structural estimators recover the ground truth to four decimal places. The neural and sieve approximation estimators recover the same ground truth at higher computational cost. The maximum causal entropy estimator returns the same parameter vector as the nested fixed point estimator, confirming the theoretical equivalence under linear features. The inverse soft Q-learning estimator recovers the parameters approximately but not exactly, reflecting the chi-squared objective rather than the maximum likelihood objective. The adversarial estimator recovers the parameters at the cost of an order of magnitude longer runtime. The behavioral cloning baseline does not estimate structural parameters but its log-likelihood is reported for comparison. All twelve estimators share the same standard error and identification pipeline, which means that the user can switch between any of them by changing one line of code.

5 Computational details

The illustrations in Section 4 were produced with **econirl** version 0.0.4 on Python 3.11. The package supports Python 3.10 or newer. The required dependencies are **JAX** 0.4.30 or newer, **Equinox** 0.11 or newer, **Optax** 0.2 or newer, **Optimistix** 0.0.7 or newer, **Lineax** 0.0.5 or newer, **NumPy** 1.24 or newer, **Pandas** 2.0 or newer, **SciPy** 1.10 or newer, and **Matplotlib** 3.7 or newer. The library is published to the Python Package Index and installs through the standard command

```
pip install econirl.
```

The development version with optional testing dependencies installs through `pip install "econirl[dev]"`. The source repository lives at https://github.com/rawatpranjal/econirl.

Estimator	θ_1	RC	log-lik.	time (s)	cos. sim.
NFXP	0.001231	3.0115	−4263.20	0.1	1.0000
CCP	0.001230	3.0113	−4263.20	0.1	1.0000
MCE IRL	0.001231	3.0115	−4263.20	0.1	1.0000
SEES	0.009546	2.9295	−4268.31	1.5	1.0000
NNES	0.031470	3.0727	−4264.00	16.0	1.0000
TD-CCP	0.001084	2.9430	−4264.97	129.6	1.0000
GLADIUS	0.001480	2.3675	−4264.45	10.4	1.0000
AIRL	0.010326	3.8970	−4467.10	607.7	1.0000
IQ-Learn	−0.066029	1.7809	−4678.13	0.0	0.9993
f-IRL	−0.000000	0.0000	−13862.95	133.8	0.0609
MPEC	0.001231	3.0115	−4263.20	0.5	1.0000
BC	—	—	−4238.07	0.0	—

Table 3: Cross-estimator benchmark on the Rust bus panel with ninety mileage states, two actions, and a discount factor of 0.9999. The columns report the operating cost coefficient, the replacement cost, the maximized log-likelihood, the wall-clock time on a single central processing unit thread, and the cosine similarity between the fitted parameter vector and the ground truth. The behavioral cloning row does not estimate structural parameters. Generated by `examples/rust-bus-engine/benchmark_all_estimators.py`.

The wall-clock numbers reported in Section 4 were measured on a single machine running macOS 14.5 on an Apple M2 Pro central processing unit with 32 gigabytes of memory. The graphics processing unit measurement for the Keane-Wolpin illustration used a single NVIDIA A100 with 40 gigabytes of memory through Google Colab. **JAX** dispatches automatically to the available accelerator, so the user code did not change between the central processing unit and the graphics processing unit run.

All scripts in Appendix A use a fixed random seed of 42 to ensure that the numbers in this paper can be reproduced exactly. Each script writes its output to a file named after the figure or table that consumes it. The full paper rebuild requires running the eleven scripts in the appendix and then recompiling the LaTeX source.

6 Summary and roadmap

econirl unifies twelve estimators for dynamic discrete choice and inverse reinforcement learning under a single result-object contract with a shared post-estimation pipeline. The shared pipeline provides four standard error methods, identification diagnostics, profile likelihood, hypothesis testing, counterfactual simulation, and visualization. Every improvement to the inference layer benefits every estimator at once. Every new estimator added to the library inherits the inference layer for free. The user picks the estimator that fits the problem and uses the same workflow regardless.

The three illustrations in Section 4 demonstrate the value of the unified workflow. The same Rust bus engine replacement panel admits both a structural likelihood fit through the nested fixed point estimator and an inverse reinforcement learning fit through the maximum causal entropy estimator. The two estimators recover the same parameter vector to four decimal places, confirming the theoretical equivalence under linear features in a setting where running both estimators required only a one-line code change. The Keane-Wolpin occupational choice panel demonstrates that the neural value approximation family scales to state spaces too large for tabular methods while preserving the unified inference workflow. The cross-estimator benchmark in Table 3 demon-

strates that all twelve estimators interoperate on the same data with the same outputs available for downstream analysis.

Three extensions are on the roadmap. Continuous-action problems will be supported through a Gauss-Hermite quadrature integration of the action space, which fits the existing protocol with minor changes to the choice probability computation. Dynamic games will be supported through a multi-agent extension that adds a best-response iteration to the existing inner loop. Bayesian inference will be supported through a Hamiltonian Monte Carlo sampler over the parameters, taking advantage of the JAX-computed gradients that the package already produces for maximum likelihood estimation. Each of these extensions preserves the result-object contract that defines the library.

econirl is open source under the MIT license. Contributions, bug reports, and feature requests are welcome at https://github.com/rawatpranjal/econirl. The package documentation is hosted at https://econirl.readthedocs.io.

Acknowledgments

We thank seminar participants at Georgetown University for valuable feedback on early versions of the package. We thank the maintainers of **JAX**, **imitation**, and **pyblp** for the open-source infrastructure that this work builds on. Any remaining errors are ours.

References

Karun Adusumilli and Dita Eckardt. Temporal-difference estimation of dynamic discrete choice models. Working paper, 2025.

Victor Aguirregabiria and Pedro Mira. Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models. *Econometrica*, 70(4):1519–1543, 2002.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. http://github.com/jax-ml/jax.

Haoyang Cao, Samuel N. Cohen, and Lukasz Szpruch. Identifiability in inverse reinforcement learning, 2021. Advances in Neural Information Processing Systems.

Christopher Conlon and Jeff Gortmaker. Best practices for differentiated products demand estimation with PyBLP. *RAND Journal of Economics*, 51(4):1108–1161, 2020.

Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. In *International Conference on Learning Representations*, 2018.

Janos Gabler and Tobias Raabe. respy: A Python package for the simulation and estimation of Eckstein-Keane-Wolpin models, 2020. https://respy.readthedocs.io.

Divyansh Garg, Shuvam Chakraborty, Chris Cundy, Jiaming Song, and Stefano Ermon. IQ-learn: Inverse soft-Q learning for imitation. In *Advances in Neural Information Processing Systems*, 2021.

Adam Gleave, Mohammad Taufeeque, Juan Rocamonde, Erik Jenner, Steven H. Wang, Sam Toyer, Maximilian Ernestus, Nora Belrose, Scott Emmons, and Stuart Russell. imitation: Clean Imitation Learning Implementations, 2022. arXiv:2211.11972.

V.J̃oseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *Review of Economic Studies*, 60(3):497–529, 1993.

Fedor Iskhakov, Jinhyuk Lee, John Rust, Bertel Schjerning, and Kyoungwon Seo. Comment on "constrained optimization approaches to estimation of structural models". *Econometrica*, 84(1): 365–370, 2016.

Enoch H. Kang, Hema Yoganarasimhan, and Lalit Jain. An empirical risk minimization approach for offline inverse RL and dynamic discrete choice model. *arXiv preprint arXiv:2502.14131*, 2025.

Kuno Kim, Shivam Garg, Kirankumar Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning, 2021. International Conference on Machine Learning.

Yong-Jun Lee, K. Sudhir, and Yifan Wang. Adversarial inverse reinforcement learning with unobserved heterogeneity, 2026. Working paper.

Yao Luo and Peijun Sang. Efficient estimation of structural models via sieves. Working paper, University of Toronto, 2024.

Hoang Nguyen. Neural networks for efficient estimation of high-dimensional dynamic discrete choice models. Working paper, Georgetown University, 2025.

Tianwei Ni, Harshit Sikchi, Yufei Wang, Tejus Gupta, Lisa Lee, and Benjamin Eysenbach. f-IRL: Inverse reinforcement learning via state marginal matching, 2021. Conference on Robot Learning.

John Rust. Optimal replacement of GMC bus engines: An empirical model of Harold Zurcher. *Econometrica*, 55(5):999–1033, 1987.

Che-Lin Su and Kenneth L. Judd. Constrained optimization approaches to estimation of structural models. *Econometrica*, 80(5):2213–2230, 2012.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, pages 1433–1438, 2008.

A Reproducing every figure and table

Every numbered figure and table in this paper is produced by exactly one script. To regenerate the paper end to end, run each script below from the repository root. All scripts use a fixed random seed of 42 and write their output to the location named in the third column.

The combined runtime of all eleven scripts is approximately one hour on the central processing unit configuration documented in Section 5, dominated by the adversarial inverse reinforcement learning estimator in Table 3. The Keane-Wolpin script accelerates by roughly a factor of five on a graphics processing unit.

Element	Script	Output
Listing 1	`code_snippets/teaser_nfxp_rust.py`	console
Table 1	`code_snippets/table1_estimator_taxonomy.py`	`figures/table1.tex`
Table 2	`code_snippets/table2_library_comparison.py`	`figures/table2.tex`
Listing 2	`code_snippets/listing2_nfxp_example.py`	console
Figure 1	`code_snippets/fig1_rust_bus_ccp.py`	`figures/fig1.pdf`
Figure 2	`code_snippets/fig2_rust_bus_value.py`	`figures/fig2.pdf`
Listing 3	`code_snippets/listing3_mce_irl_example.py`	console
Figure 3	`code_snippets/fig3_mce_irl_reward.py`	`figures/fig3.pdf`
Listing 4	`code_snippets/listing4_nnes_example.py`	console
Figure 4	`code_snippets/fig4_keane_wolpin_policy.py`	`figures/fig4.pdf`
Table 3	`examples/rust-bus-engine/benchmark_all_estimators.py`	`benchmark_results.csv`

Table 4: Mapping from each numbered element of the paper to a single reproducing script.